



• BACKEND TESTING DEEP DIVE

# Stop *Mocking* Your Database.

**SQLite** Ephemeral Testing for Node & React Backends

<> NODE.JS

REACT

SQLITE :MEMORY:

⚡ ZERO DEPENDENCIES

Integration Testing · 2026

# You wrote the tests.

*Green doesn't mean safe.*

```
expect(result).toBe("✓ passing");    # the tests passed
```

```
# meanwhile in production...
```

💥 **Production still broke.**



Passing tests confirm your assumptions — not production.

They confirm what you think happens — not what happens under real data, load, and integrations.

# The Testing Trap

You wrote the tests.  
The tests passed.  
But production still broke!



## THE MOCK ILLUSION

Your mocks return the data **you told them to return** — not what the real DB would



## INVISIBLE EDGE CASES

Edge cases in **query behavior, constraints, and joins** stay invisible until prod



## MOCK DRIFT

Teams spend hours writing mock setup code that **drifts from reality** over time

*“If your test doesn’t touch real SQL, you are just rehearsing the wrong play!”* - David Morris

# The Testing Trap: Why Most Backend Tests Lie

⚠ PAIN POINTS



## Mocks Drift From Reality

Hand-coded mock behavior diverges from actual DB logic, giving **false confidence** in tests that never touch real SQL. Your mock returns what you told it to — not what the database actually does.



## Shared Dev Databases Are Toxic

State pollution between developers and test runs causes **intermittent failures** that are impossible to reproduce. One dev's seed data is another dev's corrupted test environment.



## Full Docker DB Setups Are Slow

Spinning up a Postgres container per CI run adds **minutes of overhead** and complex orchestration. Every pipeline waits. Every developer waits. Confidence comes at an unacceptable cost.

# There Is a Better Way

## ✗ THE OLD WAY

- Mock everything.
- Hope the stubs match reality.
- Discover mismatches in production.

```
when(db.query()).thenReturn(fakeMock);  
// 💀 prod returns something else
```



Same interface.  
Real engine.

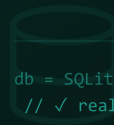
Zero infrastructure.



## ✓ THE SQLITE WAY

- Spin up a real database per test.
- Run actual SQL.
- Tear it down in milliseconds.

```
db = SQLiteDatabase.createInMemory();  
// ✓ real engine, no mocks, no lies
```



# There Is a Better Way



## TRADITIONAL APPROACHES

- ✗ Mocks that fake DB behavior
- ✗ Persistent shared dev database
- ✗ Postgres/MySQL Docker containers
- ✗ Teardown scripts that sometimes fail
- ✗ Tests that bleed state into each other

VS



## EPHEMERAL SQLITE



RECOMMENDED

- ✓ Real SQL engine, real queries
- ✓ Spins up in milliseconds
- ✓ Lives only for test duration
- ✓ Destroyed automatically on exit
- ✓ Perfectly isolated per test suite

Stop faking it – run tests against a **real SQL engine** with zero overhead

# Why SQLite for Backend Testing?



## #1

### Most Deployed Database Engine on Earth

Embedded in billions of devices, browsers, and apps worldwide



## <5ms

### Time to Spin Up an In-Memory Instance

Zero container, zero network, zero wait — instant DB on demand



## 100%

### SQL Compliant

Runs real JOINS, transactions, constraints, and triggers in your tests



## 0

### External Dependencies Required

No Docker, no daemon, no cloud credentials — ships inside Node itself

# Why SQLite?

*It's not a toy. It's the world's most deployed database.*



## #1

**Most Deployed DB on Earth**

Ships in every iOS, Android, Windows, and macOS device



## ~2ms

**Setup Time**

A fresh, isolated database spun up in under 2 milliseconds per test



## Zero

**Config Required**

No server. No credentials. No Docker.  
No ports. Just a file or `:memory:`



## Full SQL

**Standards-Compliant Engine**

JOINS, CTEs, triggers, foreign keys, indexes — all supported



SQLite passes the same **SQL compliance tests** as PostgreSQL and MySQL on core behavior.

# Zero to Real Database in 3 Steps.

*No excuses. No yak shaving.*

1



**Create** — one line, zero infrastructure

```
# Python // JavaScript
import sqlite3 const db = new Database(':memory:');
db = sqlite3.connect(':memory:')
```

2



**Migrate** — same code, same constraints

```
# Run your REAL schema migrations — nothing changes
run_migrations(db) # foreign keys, indexes, triggers — all enforced
```

3



**Tear Down** — close the connection. Clean slate guaranteed.

```
db.close() # The database is gone. Completely. No files. No state. No teardown scripts.
```

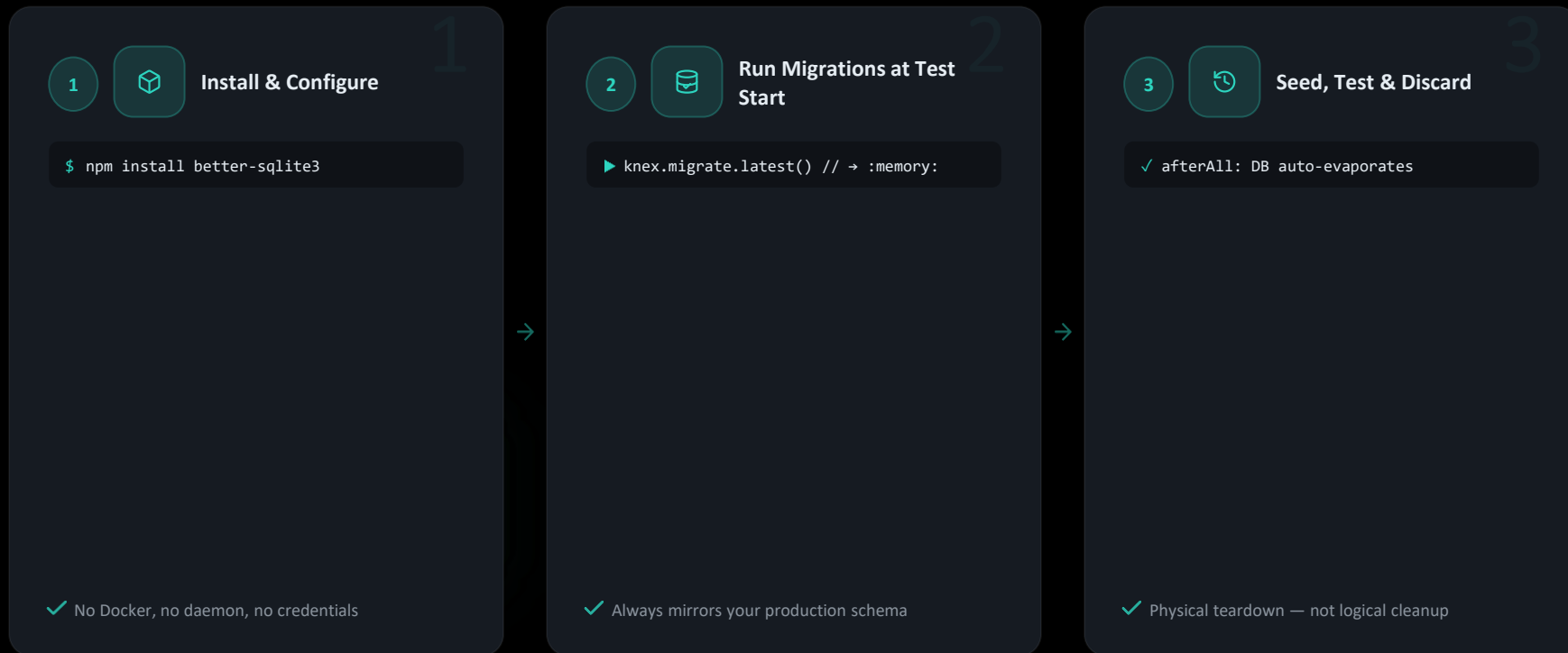


Your entire test DB lifecycle fits in a **beforeEach** / **setUp** block.

This is immediately actionable — open your editor and try it in the next 10 minutes.



# Zero to Ephemeral DB in 3 Steps



# Three Patterns That Actually Work.

✓ Pick one. Ship it this sprint.

1



## Unit-style DB Tests

One function, one DB, one assertion.

Fast, focused, surgical.

```
fn save_user() → assert rows == 1
```

• BEST FOR

Pure data-layer logic — constraints, upserts & edge cases.

2



## Integration Layer Tests

Repo → Service → DB.

No HTTP, no mocks, real SQL all the way down.

```
Repository → Service → Database
```

• BEST FOR

Multi-layer workflows where business rules touch the DB directly.

3



## Scenario / Journey Tests

Simulate a full user flow in one isolated DB. **Create**

**account** → **place order** → **verify ledger**.

```
🕒 End-to-end · One DB scope · Zero mocks
```

• BEST FOR

Critical user paths where state must flow correctly across the whole system.



All three patterns share one rule: **each test gets its own isolated database state — no shared fixtures, no bleed.**

*“The best teardown is the one you never have to write”* - David Morris

– SQLite `:memory:` databases disappear when the connection closes.



- No DELETE FROM every table
- No truncation sequences
- No transaction rollback gymnastics

Isolation is free when your database lives in RAM.

# When SQLite Is Not Enough

Know the edges so you can use the tool right.



## WHERE SQLITE SHINES

80% OF TESTS

- ✓ Unit and integration logic tests
- ✓ CI speed and isolation
- ✓ Query correctness and constraint testing
- ✓ Onboarding and local dev
- ✓ Edge case scenarios



## WHERE IT SHOWS ITS LIMITS

20% OF TESTS

- ! Postgres-specific types — JSONB, UUID, ARRAY
- ! Full-text search dialect differences
- ! Concurrent write behavior
- ! Stored procedures and DB-specific functions
- ! Production replica fidelity

Use SQLite for **80% of your tests**. Use a real Postgres container for the **20% that need it**.

# Designing Schemas That Test Well



## Surrogate IDs

01

Auto-increment or UUID primary keys prevent cross-test collision when seeding data across isolated test runs.



## Explicit Constraints

02

`NOT NULL`, `UNIQUE`, and FK constraints catch data integrity bugs that mocks silently ignore.



## Migration Files

03

Each migration is a versioned SQL file. Run them in order against the test DB at setup — no schema drift, ever.



## Factory Functions

04

Define `createUser()`, `createOrder()` helpers that return inserted rows with predictable IDs for reliable seeding.



## Per-Test Isolation

05

Spin a fresh in-memory instance per test file, or use transactions that roll back after each test for sub-millisecond reset.



## Mirror Production Schema

06

Keep test schema in sync with production migrations. Never hand-code test schemas — they drift and lie about real behavior.

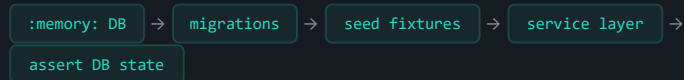


**Design principle:** A schema that tests easily is a schema that's well-modeled. Good test ergonomics and good production design are the same thing.

# Three Test Strategies for SQLite Backends

## STRATEGY 01 — RECOMMENDED DEFAULT

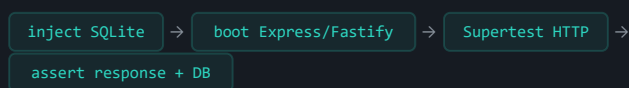
### Integration Tests



**No HTTP overhead.** Call your actual service layer functions directly. Tests your business logic against **real SQL** — JOINS, constraints, transactions — not mocked behavior.

## STRATEGY 03 — HTTP LAYER TESTING

### API Route Tests with Supertest



Assert on **response AND database state simultaneously**. Catches serialization errors, auth middleware bugs, and routing issues that pure service-layer tests miss entirely.

## STRATEGY 02 — FASTEST ISOLATION

### Transaction Rollback Pattern



Wrap each test in **BEGIN / ROLLBACK** — no re-seeding between tests. The fastest isolation strategy possible: rollback is cheaper than dropping tables, orders of magnitude faster than re-running migrations.

# Teardown That **Actually Works**



## In-Memory DB Needs No Teardown

When the Node process exits, the `:memory:` database evaporates completely. No `DROP TABLE` scripts, no leftover rows, no orphaned containers. Teardown is physical, not logical — the OS reclaims the memory. Zero cleanup burden on your test suite.



## Temp File DBs: Use OS Temp Dir

For scenarios requiring a file-based SQLite DB in tests, write to `os.tmpdir()` with a UUID filename. Register an `afterAll()` hook to call `fs.unlinkSync()` on the file path. **Reliable and cross-platform** — works identically on Linux, macOS, and Windows CI runners.



## CI Environment Guarantees

Since SQLite is bundled directly with the Node process, every CI runner — GitHub Actions, GitLab CI, Jenkins — runs your tests identically. Zero environment setup. No daemon. No credentials. **The DB is deterministic by definition** — same binary, same behavior, every time.



# Patterns From Production Teams

**PATTERN 01****DB Factory in Jest globalSetup**

01

Create one in-memory DB per worker, pass the connection string via environment variable. `globalSetup` gives you parallelism with perfect isolation.

**PATTERN 02****Knex Migrate Latest in beforeAll**

02

Call `knex.migrate.latest()` against `:memory:` in `beforeAll()`. Tests always run on the current schema — no manual migration step ever needed.

**PATTERN 03****Drizzle ORM + better-sqlite3**

03

Drizzle's synchronous SQLite driver pairs perfectly with `better-sqlite3` for zero-async overhead in unit-style integration tests. No `await` chains needed.

**PATTERN 04****Seeding With Faker**

04

Use `@faker-js/faker` to generate realistic seed data at test time. Catches edge cases that hand-crafted fixtures consistently miss in production.

**PATTERN 05****Snapshot DB State Assertions**

05

After complex ops, snapshot the whole table with `.prepare('SELECT * FROM orders').all()` and use `toMatchSnapshot()` for effortless regression detection.

**PATTERN 06****React + MSW + SQLite Backend**

06

Pair MSW (Mock Service Worker) on the frontend with a real SQLite-backed API in integration tests. True full-stack test coverage — both layers exercised simultaneously.

# Know the Limits

 Use both layers strategically



## SQLite Excels At

Recommended for these use cases

- ✓ Unit and integration test isolation
- ✓ CI speed and reliability
- ✓ Schema migration validation
- ✓ Business logic correctness
- ✓ Seeding and teardown simplicity
- ✓ Zero external dependencies

vs



## Use Production DB Parity For

When real dialect fidelity matters

- ! JSON column behavior differences (Postgres JSONB vs SQLite JSON)
- ! Full-text search dialect gaps
- ! Stored procedures and DB-specific functions
- ! Performance profiling and query plan analysis
- ! Staging and pre-production environment tests
- ! Tests that must match exact Postgres/MySQL behavior

# Take This Back to Your Codebase

- 1  **Replace mocks with real SQLite** — your tests will catch more bugs and lie less about system behavior.
- 2  **Run migrations in `beforeAll()`** — validate your schema is always in sync with production migrations.
- 3  Use `:memory:` for speed, **temp files for persistence** — choose the right mode per test scenario.
- 4  **Wrap tests in transactions for instant isolation** — rollback is faster than re-seeding.
- 5  **Layer your test strategy** — SQLite for speed and isolation, production-parity DB for staging confidence.

**15×**faster CI runs vs.  
Dockerized Postgres**0**external dependencies  
to spin up tests

## THE APPROACH

- ✓ Real SQL in every test
- ✓ Schema always validated
- ✓ Zero state pollution
- ✓ Automatic teardown
- ✓ Identical across every CI runner

Start with one test file today. Replace one mock. Measure the difference.

⚡ TAKE THIS BACK TO YOUR CODEBASE

# Your CI Pipeline Deserves Better.

One `:memory:` connection. One migration. One test that hits *real* SQL.

## YOUR NEXT STEPS — THIS WEEK

- ☐ Replace one mock with an ephemeral SQLite DB this week
- ☐ Measure your test suite time before and after
- ☐ Share the result with your team

## TALK RESOURCES



[david.morris.it@gmail.com](mailto:david.morris.it@gmail.com)

[https://github.com/towsonTigers/mockSQLite\\_demo](https://github.com/towsonTigers/mockSQLite_demo)

Slides · code samples · further reading